

Ассемблер. Лабораторная работа № 6

- 1) В программе необходимо реализовать функцию вычисления заданного условного выражения на языке ассемблера с использованием команд арифметического сопроцессора.
- 2) Результат для возврата из функции необходимо формировать в вершине стека сопроцессора.
- 3) Необходимо провести анализ полученного результата и особых состояний возникающих при вычислении.
- 4) Значения переменных передаются в качестве параметров функции.
- 5) В программе реализовать вывод результата на экран и вывод сообщения о состоянии результата.
- 6) Все параметры функции имеют тип double.
- 7) Проверку деления на 0 реализовать также на встроенном ассемблере.
- 8) В качестве комментария к каждой строке необходимо указать, какой промежуточный результат, в каком регистре формируется.
- 9) В качестве комментария к строкам, содержащим команды сопроцессора необходимо указать состояние регистров сопроцессора.

Обязательным вопросом будет являться вопрос о генерации при выполнении написанной программе особых чисел или особых состояний.

Регистр состояния SWR

Регистр SWR отражает текущее состояние сопроцессора после выполнения последней команды и состоит из:

- 6 флагов исключительных ситуаций;
- бита *SF* (*Stack Fault*) – ошибка работы стека сопроцессора. Бит устанавливается в единицу, если возникает одна из трех исключительных ситуаций (см. ниже) *PE*, *UE* или *IE*. В частности, его установка информирует о попытке записи в заполненный стек, или, напротив, попытке чтения из пустого стека. После анализа этого бита, его нужно снова установить в ноль, вместе с битами *PE*, *UE* или *IE* (если они были установлены);
- бита *ES* (*Error Summary*) – суммарная ошибка работы сопроцессора. Бит устанавливается в единицу, если возникает любая из шести перечисленных ниже исключительных ситуаций;
- четырех битов *C0-C3* (*Condition Code*) – кода условия. Назначение этих битов аналогично флагам в регистре *EFLAGS* основного процессора – отразить результат выполнения последней команды сопроцессора. В описании команд для некоторых команд сопроцессора приведена интерпретация битов *C0-C3*;
- трехбитного поля *TOP*, которое содержит номер физического регистра являющегося текущей вершиной стека.

Первые 6 флагов регистра SWR предназначены для регистрации исключительных ситуаций. Все возможные исключения сведены к шести типам, каждому из которых соответствует один бит в регистре SWR. Сопроцессор самостоятельно реагирует на эти исключения, но для этого необходимо соответствующее исключение замаскировать. Такое действие выполняется с помощью установки в единицу нужного бита в управляющем регистре сопроцессора CWR. С помощью регистра SWR фиксируются следующие типы исключений:

- *IE* (*Invalide operation Error*) – недействительная операция;
- *DE* (*Denormalized operand Error*) – денормализованный операнд;
- *ZE* (*divide by Zero Error*) – ошибка деления на ноль;
- *OE* (*Overflow Error*) – ошибка переполнения. Возникает в случае выхода порядка числа за максимально допустимый диапазон;
- *UE* (*Underflow Error*) – ошибка антипереполнения. Возникает, когда результат слишком мал;
- *PE* (*Precision Error*) – ошибка точности. Устанавливается, когда сопроцессору приходится округлять результат из-за того, что его точное представление невозможно (например, сопроцессор никогда не сможет точно разделить 10 на 3).

При возникновении любого из этих шести типов исключений устанавливается в единицу соответствующий бит в регистре SWR, вне зависимости от того, было ли замаскировано это исключение в регистре CWR или нет.

Команды сравнения данных

Команды данной группы выполняют сравнение значений числа в вершине стека и операнда, указанного в команде.

Команды сравнения данных вещественного типа

Команда	Операнды	Пояснение	Описание
FCOM FUCOM	src	ST(0) - src	Вещественное сравнение
FCOMP FUCOMP	src	ST(0) - src; TOPSWR+=1;	Вещественное сравнение с выталкиванием
FCOMPP FUCOMPP	-	ST(0) - ST(1); TOPSWR+=2;	Вещественное сравнение с двойным выталкиванием
FCOMI FUCOMI	ST, ST(i)	ST(0) - ST(i)	Вещественное сравнение с модификацией EFLAGS
FCOMIP FUCOMIP	ST, ST(i)	ST(0) - ST(i); TOPSWR+=1;	Вещественное сравнение с выталкиванием с модификацией EFLAGS
FXAM	-		Анализ ST(0)

Команды сравнения сравнивают значение в вершине стека с операндом. По умолчанию (если операнд не задан) происходит сравнение регистров ST(0) и ST(1). В качестве операнда может быть задана ячейка памяти или регистр. Команда устанавливает биты C0, C2, C3 регистра swt в соответствии с таблицей. Сбрасывает в 0 признак C1 при пустом стеке после выполнения команды.

Условие	C3	C2	C0
ST(0) > src	0	0	0
ST(0) < src	0	0	1
ST(0) = src	1	0	0
Недопустимая операция (#IA)	1	1	1

Особый интерес представляет команда FCOMI (FUCOMI). Она сравнивает содержимое регистра ST(0) со значением операнда ST(i) и устанавливает биты ZF, PF, CF регистра EFLAGS в соответствии с таблицей. Анализ выполнения сравнения осуществляет последующая команда условного перехода (команда центрального процессора).

Условие	ZF	PF	CF	Переход
ST(0) > ST(i)	0	0	0	ja
ST(0) < ST(i)	0	0	1	jb
ST(0) = ST(i)	1	0	0	je
ST(0) >= ST(i)	*	0	0	jae
ST(0) <= ST(i)	*	0	*	jbe
Недопустимая операция (#IA)	1	1	1	

Команда FXAM проверяет содержимое регистра ST(0) и устанавливает биты C0, C2, C3 регистра swt в соответствии с таблицей. Бит C1 устанавливается равным знаковому биту ST(0).

Класс	C3	C2	C0
Неподдерживаемый формат	0	0	0
Нечисло (NaN)	0	0	1
Конечное число	0	1	0
Бесконечность	0	1	1
Ноль	1	0	0
Пустой регистр	1	0	1
Ненормированное число	1	1	0

Команды сравнения данных целого типа

Команда	Операнды	Пояснение	Описание
FICOM	src	ST(0) - src	Сравнение с целым числом src
FICOMP	src	ST(0) - src; TOPSWR+=1;	Сравнение с целым числом src с выталкиванием
FTST	-	ST(0)-0;	Анализ ST(0) (сравнение с нулем)

$$\text{Пример вычисления выражения: } X = \begin{cases} b/a - 4, & \text{если } a > b; \\ 25, & \text{если } a = b; \\ (a^3 - 5)/b, & \text{если } a < b; \end{cases}$$

Анализ особенностей задачи.

Возможны две ситуации деления на ноль:

- $b/a - 4$, если $a > b$. Например, $a = 0, b = -1$.
- $(a^3 - 5)/b$, если $a < b$. Например, $a = -1, b = 0$.

В то же время ситуация, когда $a = b = 0$, является допустимой.

```
double func(double a, double b)
{
double res; int status; const int c4=4; const int c5=5; const int c25=25;
__asm{
                                st0      st1      st2      st3      st4
    finit ;                      инициализация сопроцессора
    fld  qword ptr[b];          b
    fld  qword ptr[a];          a          b
    fcom st(1); сравниваем a и b

    fstsw status; сохраняем регистр флагов сопроцессора
    mov  ah, byte ptr [status+1]
    sahf ; записываем в регистр флагов процессора

    ja  a_bigger; переход если a больше
    jb  b_bigger; переход если b больше
    ; если равны
    fild c25;                     25          a          b
    jmp  endcalc

a_bigger: ftst; сравнение a с 0
    fnstsw ax; сохраняем регистр флагов сопроцессора
    sahf ; записываем в регистр флагов процессора
    je  error; переход если a=0
    fdivp st(1), st;             b/a
    fild c4;                     4          b/a
    fsubp st(1), st;             b/a-4
    jmp  endcalc

b_bigger: fldz;                  0          a          b
    fcomp st(2);                 сравнение b с 0
    ;                             a          b
    fnstsw ax; сохраняем регистр флагов сопроцессора
    je  error; переход если b=0
    fld  st;                     a          a          b
    fmul st(1), st;              a          a*a          b
    fmulp st(1), st;             a*a*a          b
    fild c5;                     5          a*a*a          b
    fsubp st(1), st;             a*a*a-5 b
    fdiv st(1), st;              (a*a*a-5)/b
    jmp  endcalc

error:  fldz; формируем результат ошибки
endcalc: ; анализируем результат
}

}
```

Показатели работы:

Показатель	Сложность	Пояснение
Точность	2	Реализованы необходимые проверки
Оптимальность	1	Отсутствие «лишних» действий
Полнота	1	Необходимые комментарии
Оригинальность	1	Использование команд, изученных самостоятельно
Обязательный вопрос	2	
Контрольный вопрос	3	
Бонус/Штраф	1	

Контрольные вопросы к лабораторной работе № 6

1. Регистр управления
2. Регистр состояния
3. Команды сравнения данных
4. Арифметические команды сложения и вычитания
5. Арифметические команды умножения и деления
6. Дополнительные арифметические команды
7. Команды управления сопроцессором
8. Особые числа
9. Особые ситуации